

Experiment No 4

Aim: Program on Implementing Generic and HTTP servlet.

Theory Generics

in Java

Definition:

- Generics allow you to write **classes, interfaces, and methods** with **type parameters**.
- They enable **type safety** and **code reusability** without casting.
- Introduced in **Java 5**.

Why use Generics:

1. **Type Safety** – Detect errors at compile time instead of runtime.
2. **No Casting Required** – Eliminates explicit type casting.
3. **Code Reusability** – Write one method/class for multiple data types. HTTP

Servlet in Java

Definition:

- An **HTTP Servlet** is a **Java class** that handles **HTTP requests** and sends **HTTP responses** in a **web application**.
- Runs on a **Servlet container** (e.g., Apache Tomcat).
- Part of **Java EE / Jakarta EE**.

Servlet Lifecycle:

1. **init()** – Called once when servlet is created.
2. **service()** – Handles requests (called multiple times).
3. **destroy()** – Called when servlet is removed from memory.

HTTP-specific methods (in HttpServlet):

- `doGet(HttpServletRequest req, HttpServletResponse res)` → Handles GET requests.
- `doPost(HttpServletRequest req, HttpServletResponse res)` → Handles POST requests.

How it works:

1. Client (browser) sends an HTTP request.
2. Servlet container (Tomcat) calls the servlet.
3. Servlet processes request and sends an HTTP response.

Advantages:

- Platform-independent.
- Efficient and scalable.
- Works well with Java web frameworks.

Program:

1.ArithmeticServlet.java

```
package servlet;
```

```
import jakarta.servlet.ServletException; import
jakarta.servlet.annotation.WebServlet; import
jakarta.servlet.http.HttpServlet; import
jakarta.servlet.http.HttpServletRequest; import
jakarta.servlet.http.HttpServletResponse;
import java.io.IOException; import
java.io.PrintWriter;
```

```
/**
```

```
* Servlet implementation class ArithmeticServlet
```

```
*/
```

```
@WebServlet("/ArithmeticServlet") public class
```

```
ArithmeticServlet extends HttpServlet {
```

```
private static final long serialVersionUID = 1L;
```

```
/**
```

```
* @see HttpServlet#HttpServlet()
```

```
*/
```

```
public ArithmeticServlet() {
```

```
super();
```

```
// TODO Auto-generated constructor stub
```

```

    }

    /**
     * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response) */
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {

        // TODO Auto-generated method stub

        response.getWriter().append("Served at: ").append(request.getContextPath());

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String first = request.getParameter("firstno");

        String second = request.getParameter("secondno");

        String action = request.getParameter("operation");

        int x= Integer.parseInt(first);

        int y = Integer.parseInt(second);

        if("ADD".equals(action))
            out.println("Addition is: "+" "+ (x+y));                else
            if("Substraction".equals(action))
                out.println("Substraction is: "+" "+ (x-y));        else
                if("Multiply".equals(action))
                    out.println("Multiplication is "+" "+(x*y));
                else

```

```

        out.println("Division is" + " " + (x/y));

    }

    /**
     * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
     */
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {

        // TODO Auto-generated method stub

        doGet(request, response);

    }

}

```

2.ArithmeticOperations.html

```

<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

<title>Insert title here</title>

</head>

<body>

<form action="ArithmeticServlet" method="get">

    <label for="firstnumber">First Number:</label>

    <input type="text" id="firstno" name="firstno">


    <label for="secondnumber">Second Number:</label>

```

```
<input type="text" id="secondno" name="secondno">
```

```
<input type="submit" name = "operation" value="ADD">
```

```
<input type="submit" name = "operation" value="Substraction">
```

```
<input type="submit" name = "operation" value="Multiply">
```

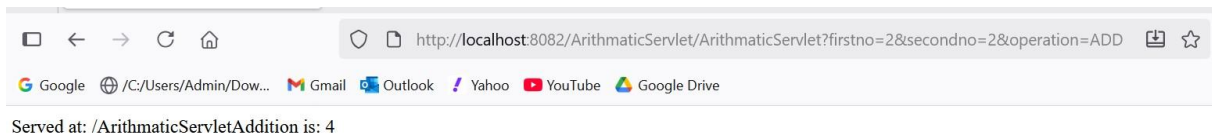
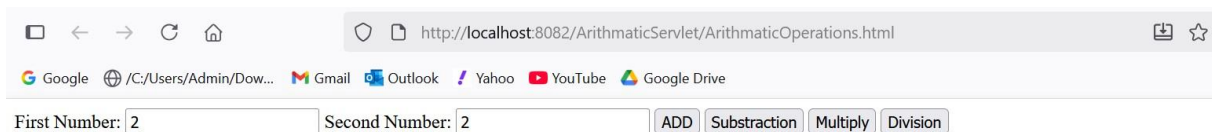
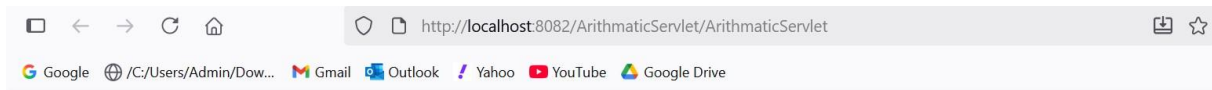
```
<input type="submit" name = "operation" value="Division">
```

```
</form>
```

```
</body>
```

```
</html>
```

Output:



Experiment No 5

Aim:

Design a login webpage in JSP that makes validation through Database using JDBC and call the servlet for various operations.

Theory

Program:

1.LoginServlet.java

```
package JSPDemo;

import jakarta.servlet.ServletException; import
jakarta.servlet.annotation.WebServlet; import
jakarta.servlet.http.HttpServlet; import
jakarta.servlet.http.HttpServletRequest; import
jakarta.servlet.http.HttpServletResponse;
import java.io.IOException; import
java.io.PrintWriter; import java.sql.*;

/**
 * Servlet implementation class LoginServlet
 */
@WebServlet("/LoginServlet") public class
LoginServlet extends HttpServlet {    private static
final long serialVersionUID = 1L;

/**
 * @see HttpServlet#HttpServlet()
```

```

    */

    public LoginServlet() {
super();

        // TODO Auto-generated constructor stub

    }

    /**

    * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)

    */

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

        // TODO Auto-generated method stub

        response.getWriter().append("Served at: ").append(request.getContextPath());

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String u_name = request.getParameter("username");

        String p_word = request.getParameter("password");

        out.println(" User is "+ u_name+" and his password is "+p_word);

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/bank",
"root", "root");

            //Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/company", "root", "root");

```

```

        String sql = "SELECT * FROM employee WHERE username=? AND
password=?";
        PreparedStatement ps = con.prepareStatement(sql);
        ps.setString(1, u_name);

        ps.setString(2, p_word);

        ResultSet rs = ps.executeQuery();
        if (rs.next())
        {
            out.println("<h2>Login Successful! Welcome, " + u_name + ".</h2>");
        } else {
            out.println("<h2>Invalid username or password!</h2>");
            //out.println("<a href='login.jsp'>Try again</a>");
        }

        con.close();    }
    catch (Exception e) {
        e.printStackTrace(out);
    }

}

/**
 * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
 */

protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

```



```
// TODO Auto-generated method stub
```

```
doGet(request, response);
```

```
}
```

```
}
```

2.login.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"    pageEncoding="UTF-8"%>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>Insert title here</title>
```

```
</head>
```

```
<body>
```

```
<%
```

```
    //String name = "Geek";
```

```
//out.println("Hello, " + name);
```

```
%>
```

```
<form action="LoginServlet" method="get">
```

```
<label for="username">User name</label>
```

```
<input type="text" name="username" id="username">
```

```
<label for="password">Password</label>
```

```
<input type="text" name="password" id="password">
```

```
<input type="submit" value="Submit">
```

```
</form>
```

```
</body>
```

```
</html>
```

Output:

Experiment No 6

Aim

Program on Implicit and Explicit objects in JSP

Theory

Implicit Objects in JSP

Definition:

- **Implicit objects** are **predefined objects** created by the JSP container.
- You can use them **directly in JSP pages without creating or importing them**.
- They make JSP coding simpler because they give direct access to request, response, session, application, etc.

List of JSP Implicit Objects:

Object Name	Type	Purpose
request	HttpServletRequest	Represents client request (form data, parameters, headers).
response	HttpServletResponse	Used to send response to client (set headers, cookies).
out	JspWriter	Writes output to client.
session	HttpSession	Stores user session data.
application	ServletContext	Stores application-wide data.
config	ServletConfig	Servlet configuration info.
pageContext	PageContext	Provides access to all JSP objects.
page	Object (this)	Refers to the current JSP page as a servlet instance.
exception	Throwable	Available only in error pages; represents error object.

Example:

```
<%  
String name = request.getParameter("username");  
out.println("Hello " + name);  
%>
```

Explicit Objects in JSP

Definition:

- **Explicit objects** are objects that **you create yourself** in the JSP page using `new` keyword or by calling a method.
- Not automatically provided by the container.
- Used when you need custom functionality or data storage.

Example:

```

<%
    // Explicit object creation    java.util.Date
today = new java.util.Date();
out.println("Today's Date: " + today);
%>

```

Key Difference Between Implicit and Explicit Objects

Feature	Implicit Objects	Explicit Objects
Created by	JSP Container	Programmer
Availability	Available by default	Must be explicitly created
Examples	request, response, session	new Date(), custom class objects
Purpose	To simplify JSP coding	To perform specific, custom tasks

Program:

```

<%@ page language="java" contentType="text/html; charset=UTF-8"    pageEncoding="UTF-8"%>

<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

<title>Implicit and Explicit</title>

</head>

<body>

<%
    String name = request.getParameter("username");
out.println("Hello, " + name);
%>

<br>

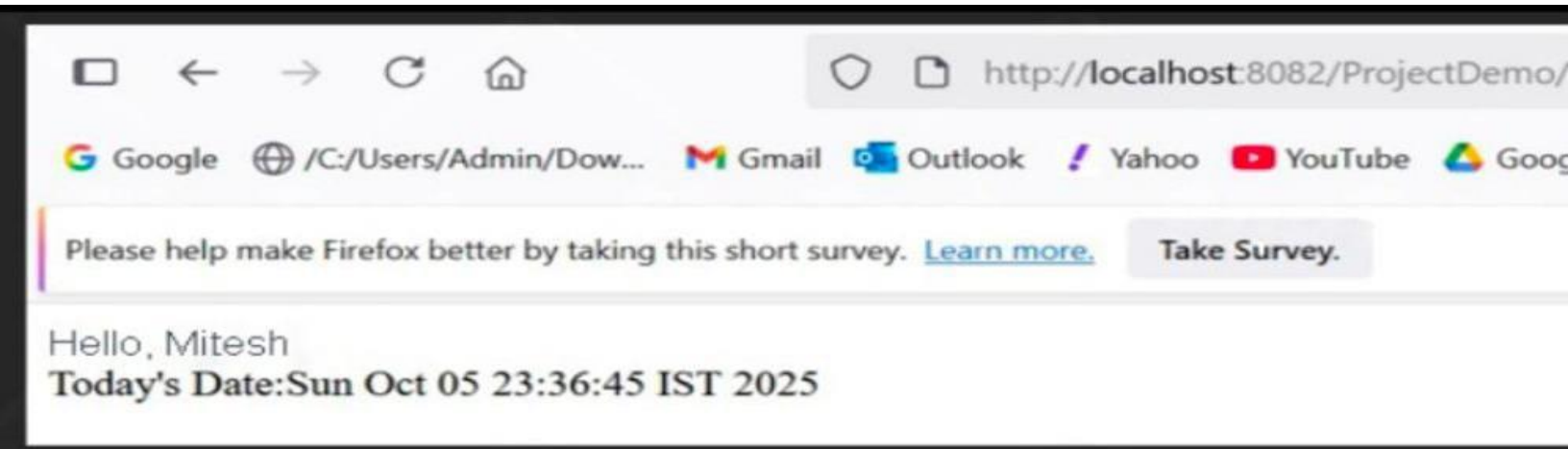
<%java.util.Date today=new java.util.Date();
out.println("Today's Date:"+today);
%>

</body>

```

</html>

Output:



Experiment No 7

Aim

Program using Java Script to validate the email address entered by the user (check the presence of “@” & “.” character. If this character is missing, the script should display an alert box reporting the error and ask the user to re-enter it again).

Theory

Program

```
<!DOCTYPE html>

<html>

<head>

<title>Email Validation</title>

<script>

function validateEmail() {

let email = document.getElementById("email").value;

if (email.includes("@gmail.com")) {

alert("Great! Your Gmail address looks valid: " + email);

return true;

} else {

alert("Invalid Email! Your email must contain '@gmail.com'. Please try again.");

document.getElementById("email").value = "";

return false;

}

}

</script>

</head>

<body>

<h2>Email Validation using JavaScript</h2>
```

```
<form onsubmit="return validateEmail()">
```

Enter Email:

```
<input type="text" id="email" placeholder="example@gmail.com">
```

23

```
<input type="submit" value="Check Email">
```

```
</form></body></html>
```

Output

Email Validation using JavaScript

Enter Email: 23

127.0.0.1:5500 says

Great! Your Gmail address looks valid: abc@gmail.com

OK

